

[🏠 Home](#) > [Categories](#) > [Embedded](#) >

Embassy——易用的嵌入式Rust异步框架

2023-01-25

► Table of contents

有些「小制作」需要用到USB、蓝牙、WiFi，我又想用 Rust 进行编程，但很多 Crates 提供的方法很有限。目前我粗略接触过的 ``esp-idf-sys``、``Tock OS`` 之类，都缺少了一些我计划中需要的关键功能（这么看感觉 C 那边生态优势很明显，瞧瞧 ``Zephyr RTOS`` 的成熟度）。最近在搜索 Rust 可用的蓝牙协议栈时，看到了 ``Embassy`` 这个框架，第一感觉是帮我简化了很多 MCU 外设管理的冗杂操作，顿时留下不错的印象。下面是一些个人学习记录。

之前觉得是给自己留的坑，写了一半没继续，如今看来还能有点用处，先挂出来吧。写得不咋地，还可能有错，见谅。

Embassy版本更新会带来一些API变更，建议从官方master分支的example开始建立代码工作区。本文不一定适用。

Embassy 点灯🔦

废话不多说，先点个灯，好有个直观感受。嘛，还是得先配置好开发环境。先简单列出我的开发资源配置。

- 笔记本电脑，操作系统为 Arch Linux
- 开发板：WeAct BluePill Plus (STM32F103CBT6)
- 调试器：Muse Lab 高速 DAPLink (兼容公版 ATSAM3U2C DAPLink)

首先准备好 Rust 工具链，这个可以参考 ``https://rustup.rs`` 提供的信息进行安装，此处不再详叙。

由于我们的目标板 CPU 架构为 ARM Cortex-M，需要额外安装对应的 *target*。不过幸运的是，embassy 项目空间内已经写好了工具链配置 (``rust-toolchain.toml``)，待会跑 blinky 的时候

rustup 会自动处理工具链依赖。

这边编程也 **不用** 多数人熟悉的 OpenOCD 了（尽管也可以），使用的是基于 Rust Crate `probe-rs` 的一系列工具。这里主要使用的是 *probe-run*¹（这样可以直接跑官方例程）。使用下面的指令安装 *probe-run*。

```
# 安装 probe-run
cargo install probe-run
```

关于 *probe-run* 的配置，待进入示例项目再看。

如果 rustup、cargo 下载的速度过慢

可以尝试一下配置使用其它镜像或加速站点^{2 3 4}。需要额外注意的是，如果自己的 git 配置文件（`~/.gitconfig`）中配置了代理，建议在操作 rustup 和 cargo 的过程中删除代理相关配置。

开发板选择参考

目前 Embassy 对 STM32、NRF52 的支持情况足够好，资源丰富，所以选相关开发板能省不少力。不过要是有余力，也可以去看看 Xtensa（ESP32）或者 RISC-V（ESP32-C3）架构的支持，截至 2023-01-26，Embassy 已经有对它们的初步支持，而相关工具（*probe-run*）也支持了 ESP32-C3 内置的 JTAG 适配器。

准备运行示例项目，先准备一份源代码，操作如下：

```
git clone --depth 1 --recursive https://github.com/embassy-rs/embassy
```

随后进入适用于自己开发板 MCU 的示例项目文件夹。

```
cd embassy/examples/stm32f1
```

看一下示例项目内有哪些材料：

```
> tree -a
```

```
.
```

```
├─ build.rs
```

```
# 最早编译并运行的 build script
```

```

└─ .cargo
  └─ config.toml      # cargo 配置, 关于 probe-run 的配置也在这里
└─ Cargo.lock         # 锁, 锁的是 Crates 版本
└─ Cargo.toml         # 项目配置
└─ src                # 这里有 4 个示例程序
  └─ bin
    └─ adc.rs
    └─ blinky.rs
    └─ hello.rs
    └─ usb_serial.rs

```

这次就跑个 blinky (`blinky.rs`), 检查下代码适不适合我的板子, 一看, 就 LED 引脚不一样。那就简单修改一下。

```

-   let mut led = Output::new(p.PC13, Level::High, Speed::Low);
+   let mut led = Output::new(p.PB2, Level::High, Speed::Low);

```

再检查一下 probe-run 的配置 (在 `.cargo/config.toml`) 和我的 MCU 是否一致。一看, 倒也不用改。

```

[target.'cfg(all(target_arch = "arm", target_os = "none"))']
# replace STM32F103C8 with your chip as listed in `probe-run --list-chips`
- runner = "probe-run --chip STM32F103C8"
+ runner = "probe-run --chip STM32F103CB"

[build]
target = "thumbv7m-none-eabi"

[env]
DEFMT_LOG = "trace"

```

接下来就连接好开发板, 执行下面的指令编译、刷入固件。

```
cargo run --bin blinky
```

等编译、刷写完毕, 应该就能看到板子上的 LED 在闪烁了。

关于 LED 闪烁时间准确性

你可能会觉得闪烁时间和程序里写的 300 ms 差别有点大，我也不知道为什么。Debug 日志输出使用了RTT，应该影响不大；由此来看很可能是因为 Embassy 的非抢占式调度。谈起调度，可以了解下 *RTIC*，Embassy 和 RTIC 的关系可以说是各自的重点不同⁵。

Embassy 设计初探⁶

Embassy 以一个 Executor 为核心。所有的任务用一个宏标记。以 `blinky.rs` 为例：

```
#![no_std]
#![no_main]
#![feature(type_alias_impl_trait)]

use defmt::*;
use embassy_executor::Spawner;
use embassy_stm32::gpio::{Level, Output, Speed};
use embassy_time::{Duration, Timer};
use {defmt_rtt as _, panic_probe as _};

#[embassy_executor::main]
async fn main(_spawner: Spawner) {
    let p = embassy_stm32::init(Default::default());
    info!("Hello World!");

    let mut led = Output::new(p.PB2, Level::High, Speed::Low);

    loop {
        info!("high");
        led.set_high();
        Timer::after(Duration::from_millis(300)).await;

        info!("low");
        led.set_low();
        Timer::after(Duration::from_millis(300)).await;
    }
}
```

在 blinky 的例子中，只是用宏标记了一个 `main` 入口点（此处已简化许多），Embassy 在完成一定初始化后会从这里开始执行主程序。但是这样好像同时只有一个任务在运行？也确实如此，毕竟还有一个 `Task` 没有引入。来看官方提供的另一个例子。

```

#![feature(type_alias_impl_trait)]

use embassy_executor::Spawner;
use embassy_time::{Duration, Timer};
use log::*;

#[embassy_executor::task]
async fn run() {
    loop {
        info!("tick");
        Timer::after(Duration::from_secs(1)).await;
    }
}

#[embassy_executor::main]
async fn main(spawner: Spawner) {
    env_logger::builder()
        .filter_level(log::LevelFilter::Debug)
        .format_timestamp_nanos()
        .init();

    spawner.spawn(run()).unwrap();
}

```

此处便可以看到，通过 Embassy 提供的宏，创建了一个 Task，而通过调用这个 Task，便可以得到一个 task 实例（实际上和 coroutine 差不多）。有过异步编程经验的人应该很快就能认出这个简单的创建多个任务的模式（个人感觉和 CircuitPython 的使用逻辑很接近）。

回到之前所说的，Embassy 的核心是一个 Executor。根据官方文档²的介绍：

“executor 维护一个存储需要轮询（poll）的任务队列。当一个任务（task）创建时，它就被检视（poll）一次。这个任务会一直执行到它被阻塞。这可能发生在任何等待（await）async 函数返回结果的时候。而当前述事件发生时，对应任务会暂停执行（yield）并返回 `‘Poll:Pending’`。随后，executor 将该任务放在任务队列尾部，去检视下一个任务。如果一个任务完成或者取消了，它就不再入队。”

当然，这也有个前提：没有哪个任务会一直阻塞、不提供切换任务的机会。不然 executor 将永远无法切换任务，整个系统也退化成了单道程序处理机。

1. [Crates.io 上的 probe-run ↩](#)
2. [USTC Rust 工具链反向代理配置说明 ↩](#)
3. [USTC Crates 源使用帮助 ↩](#)
4. [Rsproxy 主页 ↩](#)
5. [Should I use Embassy or RTIC or both \(for embedded rust\)? – James Munns ↩](#)
6. [Embassy layer by layer ↩](#)
7. [Embassy executor ↩](#)

embassy rust

« PREV PAGE

NEXT PAGE »

用 Embassy 开发 Xinput 协议手柄

蓝牙 SBC 双通道高清音频模式